

Trabalho Pratico 2 -- Pre-processamento e Analise Preditiva

Disciplina: Inteligencia Artificial Aluno: Joao Victor Borges Carvalho Dataset: Breast Cancer Wisconsin (Original) -- UCI Machine Learning Repository (id=15)

Sobre o projeto

Este trabalho implementa um pipeline completo de pré-processamento e análise preditiva sobre a base Breast Cancer Wisconsin, com 699 amostras de tecido mamário classificadas como benignas ou malignas a partir de 9 características celulares avaliadas por patologistas.

Toda a parte matemática e todos os algoritmos de aprendizado de máquina foram implementados do zero, usando apenas Python puro e o módulo math. Nenhuma biblioteca com fórmulas prontas foi utilizada (scikit-learn, pandas para cálculos, numpy, scipy). A única exceção é o matplotlib, usado exclusivamente para plotagem dos gráficos.

Estrutura do projeto

```
trabalho2IA/
├── data/
│   └── breast_cancer_wisconsin.csv # Dataset original (699 linhas x 10 colunas)
├── plots/
│   ├── boxplots_por_classe.png      # Boxplots comparativos benigno vs maligno
│   ├── distribuicao_classes.png      # Distribuicao das classes (barras)
│   ├── histogramas_por_classe.png   # Histogramas sobrepostos por classe
│   └── mlp_convergencia.png          # Curva de convergência da MLP
├── utils.py                          # Biblioteca de funções matemáticas e algoritmos
(~1.160 linhas)
├── main.py                          # Pipeline completo (script unico, ~1.200 linhas)
├── 01_analise_exploratoria.ipynb    # Notebook 1: Etapas 1 a 5 (exploração)
└── 02_preprocessamento.ipynb       # Notebook 2: Etapas 6 a 10 (limpeza e
normalização)
```

|— 03_modelagem_preditiva.ipynb # Notebook 3: Etapas 2.1 a 2.7
(algoritmos)
|— documentacao_final.md # Documentação detalhada com justificativas
|— apresentacao.pptx # Slides
|— README.md # Este arquivo

Dataset

O Breast Cancer Wisconsin (Original) possui 699 amostras de tecido mamário. Cada amostra tem 9 features, todas notas de 1 a 10 atribuídas por patologistas:

Feature	Descrição
Clump_thickness	Espessura do aglomerado
Uniformity_of_cell_size	Uniformidade do tamanho celular
Uniformity_of_cell_shape	Uniformidade do formato celular
Marginal_adhesion	Adesão marginal
Single_epithelial_cell_size	Tamanho de célula epitelial única
Bare_nuclei	Núcleos nus (16 valores ausentes)
Bland_chromatin	Cromatina suave
Normal_nucleoli	Nucléolos normais
Mitoses	Taxa de divisão celular

Target: Class -- 2 (benigno) ou 4 (maligno), convertido para 0/1.

Pipeline -- Parte 1: Pré-processamento

Etapa	Resultado
1. Atributo alvo	Class convertido: 2 -> 0 (benigno), 4 -> 1 (maligno)
2. Tipos de dados	Todos Quantitativos Discretos
3. Escala	Intervalar Discreta
4. Estatísticas	Média, mediana, moda, variância, IQR + histogramas e boxplots
5. Split treino/teste	Hold-out 80/20 estratificado (seed=42)
6. Eliminação	0 colunas removidas, 184 duplicatas removidas
7. Desbalanceamento	1.09:1 -- dataset balanceado
8. Limpeza	Outliers mantidos; 0 inconsistentes; 0 ausentes
9. Normalização	Min-Max [0, 1]
10. Dimensionalidade	Correlação de Pearson -- 9/9 features mantidas

Resultado final: 362 exemplos de treino x 9 features normalizadas, 137 exemplos de teste.

Pipeline -- Parte 2: Análise Preditiva

Algoritmos implementados do zero

Baseline: Classe majoritária (sempre prediz a classe mais frequente)

K-NN: Distância euclidiana + votação majoritária (K=3, impar para evitar empate)

Árvore C4.5: Gain Ratio (Information Gain / Split Info) + poda por profundidade

MLP: Rede neural 9->8->1 com sigmoide, backpropagation manual (regra da cadeia), SGD, 500 épocas

Métricas implementadas do zero

Matriz de confusão (TP, TN, FP, FN), acurácia, precisão, recall (sensibilidade), especificidade, F1-score.

Classe positiva = 1 (Maligno). Recall é a métrica mais crítica no contexto médico (falso negativo = câncer não detectado).

Resultados no conjunto de teste

Tabela 1 – Resultados dos modelos no conjunto de teste

Modelo	Acurácia	Precisão	Recall	F1-Score
Baseline	0.3504	0.3504	1.0000	0.5189
K-NN (K=3)	0.9781	1.0000	0.9375	0.9677
C4.5 (d=3)	0.9708	0.9783	0.9375	0.9574
MLP (8 ocultos)	0.9635	0.9778	0.9167	0.9462

Fonte: Elaborado pelo autor (2026)

K-NN com K=3 foi o melhor modelo (F1=0.9677), com precisão perfeita -- zero falsos positivos.

A C4.5 com profundidade 3 entrega resultado quase idêntico (F1=0.9574) com apenas 15 nós, sendo totalmente interpretável. A raiz da árvore e Uniformity_of_cell_size ≤ 0.2778 .

A MLP teve uma única configuração de hiperparâmetros testada (limitação de tempo), então o F1 de 0.9462 e um limite inferior do que a rede pode alcançar com tuning adequado.

Decisões e justificativas

Escala intervalar, não racional: Não há zero absoluto verdadeiro -- nota 1 e o piso da escala, não ausência total. Mas a literatura médica trata como intervalar para permitir estatística paramétrica.

Moda para imputação, não média: Dados são discretos (1-10). A média geraria valores decimais inexistentes na prática clínica (ex: 3.7).

Outliers mantidos: No contexto médico, nota 10 não é erro de medição -- e sinal de alta severidade. Removê-los eliminaria os casos mais informativos.

Min-Max [0,1], não Z-score: Escala original já é limitada (1-10), o Min-Max preserva a estrutura. Essencial para K-NN (features com maior magnitude dominariam a distância) e MLP (evitar saturação da sigmoide).

Correlação de Pearson, não PCA: Implementável do zero com Python puro. PCA exigiria decomposição em autovalores -- mais complexa e com perda de interpretabilidade.

Hold-out 80/20 estratificado, não cross-validation: Com 3 algoritmos implementados manualmente, 5-fold CV seria muito custoso. O ganho marginal não justifica para 362 exemplos.

K=3 (ímpar): Evita empate na votação binária. K=1 sofre overfitting; K>=5 suaviza demais a fronteira de decisão.

Gain Ratio, não só Information Gain: O IG puro favorece features com muitos valores distintos. O Gain Ratio corrige esse viés dividindo pelo Split Info.

Como executar

Requisitos

Python 3.10+

matplotlib

jupyter (opcional, para os notebooks)

pip install matplotlib jupyter

Opção 1: Jupyter Notebooks (recomendado)

jupyter notebook

Execute na ordem: 01_analise_exploratoria.ipynb -> 02_preprocessamento.ipynb -> 03_modelagem_preditiva.ipynb

Cada notebook é autocontido -- se quiser pular direto para o 3, ele recarrega os dados do zero e reproduz todo o pipeline.

Opção 2: Script único

```
python main.py
```

Executa o pipeline completo e imprime todos os resultados no terminal. Leva cerca de 30-60 segundos (a MLP e a parte mais lenta).

Implementação from scratch

Todas as funcoes abaixo foram implementadas manualmente em utils.py (Python puro + math):

Estatística: média, mediana, moda, variância (amostral), desvio padrão, mínimo, máximo, amplitude, percentil com interpolação linear, quartis, IQR, correlação de Pearson.

Pré-processamento: contador de frequência manual, stratified split com LCG próprio (Fisher-Yates determinista), detecção de outliers (IQR), imputação pela moda, remoção de colunas constantes, remoção de duplicatas, normalização Min-Max.

Métricas: matriz de confusão (TP/TN/FP/FN), acurácia, precisão, recall, especificidade, F1-score.

Algoritmos: baseline (classe majoritária), K-NN (distância euclidiana, votação), C4.5 (entropia, Information Gain, Split Info, Gain Ratio, árvore recursiva com poda), MLP (forward pass, backpropagation com regra da cadeia, SGD, inicialização de pesos).